백엔드 실무 지식 · 11장

SERVER STRUCTURE & PATTERNS

# 자주 쓰는 서버 구조와 설계 패턴

아키텍처 수준의 여섯 가지 패턴을 **개념 정의**와 **처리 흐름** 중심으로 정리합니다.

MVC | 계층형 | DDD | MSA | EDA | CQRS

이 장의 구성

# 여섯 가지 챕터

서버 애플리케이션을 개발하다 보면 반복되는 구조와 패턴이 있습니다. 이 장의 주제는 아키텍처 수준에서 자주 등장하는 **MVC · 계층형 아키텍처 · DDD와 전술 패턴 · 마이크로서비스(MSA) · 이벤트 기반(EDA) · CQRS**입니다.

01

# MVC 패턴

Model · View · Controller — 역할 분리와 요청·응답 흐름

01 · 개념과 원칙

# 세 구성 요소와 MVC가 지키는 규칙

### Model

회원 가입·암호 변경 등 **비즈니스 로직**만 처리. 화면이나 UI 흐름 제어는 하지 않음.

### View

사용자에게 보여 줄 **결과 표현**(HTML·JSON 등). 로직·흐름 제어는 담당하지 않음.

### Controller

**입력 해석**, 모델 호출, **어떤 뷰로 응답할지 선택**. 검증·인증 등 횡단 관심도 여기에 두는 경우가 많음.

### 의존 방향

모델과 뷰는 **서로 의존하지 않음**. 컨트롤러만 모델·뷰를 알고 조율 → 교체·테스트가 쉬움.

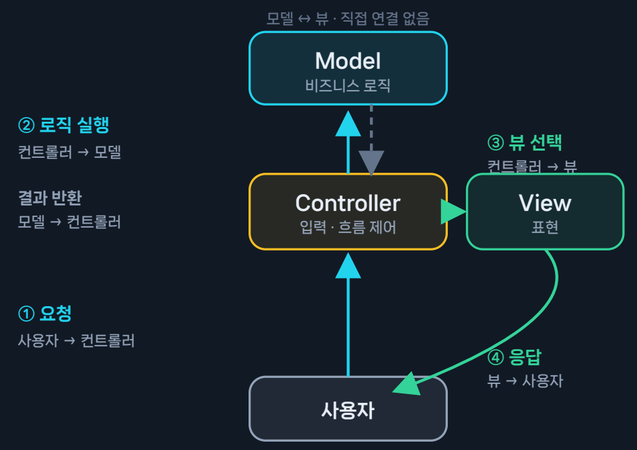
**핵심** ① 비즈니스(모델)와 표현(뷰)을 분리합니다. **핵심** ② 흐름과 사용자 입력 처리는 컨트롤러에 모읍니다.

01 · 흐름

# 요청이 한 바퀴 도는 순서

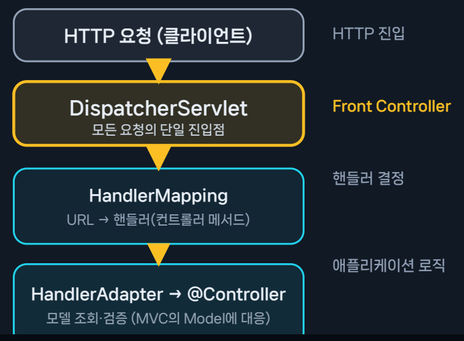
그림 11.1과 같이 사용자는 컨트롤러에 요청을 보내고, 컨트롤러가 모델에 로직 실행을 위임한 뒤 결과로 뷰를 고릅니다. 뷰가 응답을 만들며, 모델과 뷰는 서로 직접 연결되지 않습니다. 오른쪽은 Spring에서 **DispatcherServlet**이 그 역할을 어떻게 나누어 처리하는지 대응 관계입니다.

패턴 · MVC 연결 관계




모델과 뷰 사이에는 데이터 흐름 화살표가 없습니다. 모든 조율은 컨트롤러를 통합니다.

구현 · SPRING WEB MVC



02

# 계층형 아키텍처

역할 분담 · 하위 계층만 의존

02 · 계층 정의와 규칙

# 표현 → 응용 → 도메인 → 인프라

계층형 아키텍처(Layered Architecture)는 **계층마다 특정 역할을 수행**하고, **자기보다 하위에 위치한 계층에만 의존**하는 특징을 가집니다. 아래쪽 계층은 위쪽을 알지 못하므로, 상위가 하위를 호출하는 단방향으로 호출 흐름이 정리됩니다.

규칙이 단순해 실행 경로를 따라가기 쉽다는 장점이 있습니다.

**표현 (Presentation / UI)**
사용자와의 상호작용. MVC의 컨트롤러·뷰가 여기에 해당. 요청을 응용에 넘김.

**응용 (Application)**
유스케이스 조율. "서비스" 계층. 도메인·인프라를 호출해 결과를 표현에 반환.

**도메인 / 모델**
비즈니스 규칙·상태 전이. 주문 취소 조건 등이 이쪽에 모이는 것이 이상적.

**인프라 / 영속**
DB·메시지·외부 API 등 기술 세부. DAO·클라이언트가 대표적.

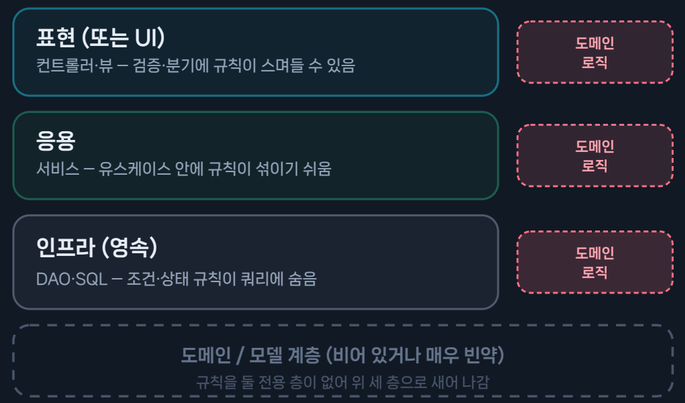
**Layer(논리)**는 코드 속 책임 묶음이고, **Tier(물리)**는 배포 단위입니다. 같은 논리 계층이라도 한 프로세스에 둘 수도, 서버를 나눌 수도 있습니다. 호출 규칙으로 **Strict**(꼭 인접 계층만)는 예측 가능·낮은 결합에, **Relaxed**(건너뛰기 허용)는 구현·지연 측면 이점과 **결합도·지연 트레이드오프**를 동시에 안고 갑니다.

02 · 그림 11.4

# 도메인 계층이 비면 로직이 퍼집니다

도메인 계층이 거의 없을 때는 표현·응용·인프라만 남는 **사실상 세 계층**에 가깝습니다. 그러면 회원 상태 전이 같은 **도메인 로직**이 컨트롤러·서비스·SQL 등으로 갈라져, "어떤 조건에서 상태가 바뀌는지"를 보려면 쿼리와 여러 클래스를 동시에 뒤져야 합니다. 이것이 도메인 모델이 빈약할 때 나타나는 **로직 흩어짐**입니다.

그림 11.4 · 도메인 로직이 각 계층으로 퍼지는 경향




아래는 `MemberDao.updateMemberStatus(id)`에 대응하는 전형적인 예입니다. 코드만 보면 도메인 로직이 거의 없고, **어떤 조건에서 상태가 바뀌는지**는 SQL을 열어봐야만 알 수 있습니다.

SQL

```sql
-- MemberDao.updateMemberStatus(id)
UPDATE member
SET status = 20
WHERE member_id = ? AND status = 10;
```

호출 코드

```java
int cnt = mdao.updateMemberStatus(id);
if (cnt == 0) {
    // 변경 건이 없으므로 변경 실패 처리
}
```

02 · 계층형과 비교

# 계층형 vs 헥사고날(포트·어댑터)

계층형은 **표현 → 응용 → 도메인 → 인프라**로 호출이 정리되지만, 관성적으로 **도메인이 인프라(DB·외부 API 구현체)에 직접 의존**하기 쉽습니다. 그러면 앞서 본 것처럼 **규칙이 SQL·매퍼 코드로 새 나가는** 일이 잦아집니다. **의존성 역전(DIP)**으로 "추상(포트)은 도메인 쪽, 구현(어댑터)은 바깥"에 두는 습관이 필요합니다.

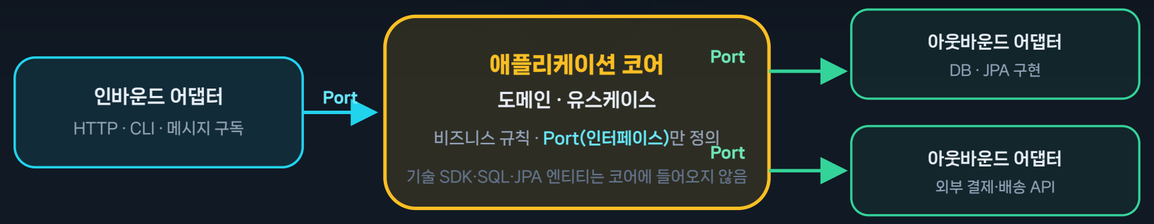
**헥사고날**은 **도메인·유스케이스를 코어**에 두고, HTTP·메시지 등 진입은 **인바운드(주도) 어댑터**, DB·외부 API는 **아웃바운드(피동) 어댑터**로 두어 **기술 세부에 도메인이 끌려가지 않게** 합니다. 핵심 한 줄은 **코어(도메인)는 DB·외부 API 같은 기술을 "모른다"**는 것입니다. DDD가 말하는 **"도메인 로직을 모델 안에 모으기"**를 구조적으로 뒷받침하는 대표적인 방향 중 하나입니다.

### 전형적 계층형의 함정

- 도메인이 Repository 구현·ORM·쿼리에 직접 묶임
- 인프라 변경이 도메인 코드까지 흔듦
- DIP 없이 "아래 계층"만 바꾸기 어려움

### 헥사고날의 초점

- 코어는 포트(인터페이스)만 알고 구체 기술은 몰라도 됨
- DB·REST·메시지는 전부 교체 가능한 어댑터
- 테스트 시 어댑터를 가짜로 바꾸기 쉬움




외부는 어댑터로만 붙고, 코어는 "무엇을 할지"만 유지

의존성 방향: 어댑터 → 포트(코어가 정의) ← 구현은 바깥

헥사고날은 형태만 육각형이 아니라, 도메인을 중심에 두고 기술을 가장자리로 밀어내는 의존성 규칙이 핵심입니다.

03

# DDD와 전술 패턴

도메인 로직을 모델 안에 모으기

# DDD에서 도메인 모델의 구성요소

**엔티티/밸류** · Entity · Value

엔티티는 식별자로 구분·추적(상태가 바뀌어도 같음). 밸류는 식별자 없는 값(금액·주소 등), 보통 불변.

**애그리거트** · Aggregate

관련 객체를 하나의 개념 단위로 묶습니다. 이 안에서 규칙·일관성을 관리합니다.

**리포지토리** · Repository

도메인 객체와 DB 등 저장소 사이를 연결합니다. 저장·조회는 애그리거트 단위로 다룹니다.

**도메인 서비스** · Domain Service

한 애그리거트에 넣기 애매한 규칙, 외부 연동이 필요한 도메인 로직을 둡니다.

**도메인 이벤트** · Domain Event

도메인에서 일어난 사건. 상태 변경 시 발생하고, 다른 부분에 알릴 때 씁니다.

RDB **중심 설계**는 테이블을 **외래 키(FK)**로 촘촘히 연결하려는 경향이 있습니다. **DDD**에서는 **애그리거트 간에는 객체 참조 대신 ID로만 참조**해 경계를 분명히 하고, 트랜잭션으로 묶을 강한 일관성은 **애그리거트 안**에서만 책임집니다. 애그리거트 밖의 규칙은 이벤트·API 등으로 맞춥니다.

**Bounded Context**는 패키지 경계가 아니라 **같은 용어·같은 모델 규칙이 통하는 언어적 경계**입니다. **Ubiquitous Language**는 전문가와 개발자가 쓰는 말을 클래스·API·문서에 그대로 반영해 모델을 정렬합니다. 컨텍스트마다 동일한 단어라도 의미가 다를 수 있으므로, 경계 밖과는 명시적으로 통합 전략을 짜야 합니다.

04

# 마이크로서비스

작은 배포 단위와 도메인 경계

04 · 가치와 트레이드오프

# 모놀리식 vs 마이크로서비스

핵심 가치는 **독립적 배포**와 **도메인을 기준으로 한 경계**입니다. 한 도메인이 여러 서비스에 흩어지면 출시·조율 비용이 커집니다.

### 모놀리식

- 배포·코드 탐색 단순, 통신·운영 단순
- 규모가 커질수록 변경 충돌·장애 반경·배포 단위가 커짐

### 마이크로서비스

- 팀·기능 단위 배포·확장, 기술 다양성
- 분산 추적·테스트·데이터 일관성·네트워크 비용

구조가 나쁜 모놀리식을 잘게 쪼개도 품질이 낮으면 **분산 모놀리식**이 될 수 있습니다. 한 프로세스 안에서라도 **모듈러 모놀리식**으로 경계를 세워 응집도를 높이는 접근이 있습니다.

**콘웨이의 법칙(Conway's Law)**을 떠올리면, 아키텍처는 기술 스택만의 문제가 아니라 **조직의 소통·책임 구조**와 맞물립니다. 팀이 나뉘는 방식이 곧 서비스 경계에 스며들기 쉬우므로, MSA를 논할 때 **"누가 무엇을 소유하고 어떻게 합의하는가"**를 함께 봐야 합니다.

04 · 구조를 풀어 쓰기

# 마이크로서비스 6가지 핵심 개념

1. **독립적 배포**
   다른 마이크로서비스를 배포하지 않고도 이 서비스만 변경·배포·출시할 수 있어야 합니다.

2. **도메인을 중심으로 모델링**
   각 마이크로서비스는 도메인을 기준으로 나뉩니다. 한 도메인의 기능이 여러 서비스에 흩어지면 출시·조율 비용이 커집니다.

3. **자신의 상태를 가짐**
   서비스끼리 DB를 공유하지 않습니다. 다른 서비스 데이터가 필요하면 DB에 직접 붙지 말고 API 등으로 접근합니다.

4. **크기**
   서비스 크기에 만능 기준은 없습니다. 조직이 감당할 수 있는 단위와 경계 정의에 맞춥니다.

5. **유연함**
   기술·확장·견고함 같은 유연함을 얻는 대신 비용이 듭니다. 감당 가능한 범위에서 도입 여부를 봅니다.

6. **아키텍처와 조직을 맞춤**
   「조직 구조는 아키텍처에 영향을 준다」는 콘웨이 법칙에 가깝습니다. 비즈니스 도메인이 시스템 아키텍처를 주도하는 원동력이 되도록 설계합니다.

04 · 모놀리식 오해

# 음, 모놀리식이 나쁜 건가?

교재 15장 발췌

잘못된 것은 **모놀리식**이라는 형태가 아니라 **설계·코드 품질**인 경우가 많습니다. 품질을 고치지 않고 나누기만 하면 **분산 모놀리식**으로 복잡도만 커질 수 있습니다.

그래도 모놀리식을 쓴다면 **모듈러 모놀리식**처럼 모듈 경계를 분명히 해 의존을 줄이는 편이 낫습니다.

05

# 이벤트 기반 아키텍처

사실(Event)을 축으로 한 비동기 연결

05 · 구성과 책임

# 생산자 · 브로커 · 소비자

이벤트는 "이미 일어난 사실"을 과거형으로 표현합니다. 컴포넌트는 직접 호출 대신 **브로커**를 사이에 두어 **느슨한 결합**과 독립 배포를 얻습니다. 스키마·순서·재시도 시나리오에 맞춰 **메시지 설계와 멱등 처리**를 함께 맞춥니다.

발행된 이벤트는 브로커(라우터)에서 나뉘어, 관심 있는 소비자들에게 비동기로 전달됩니다.

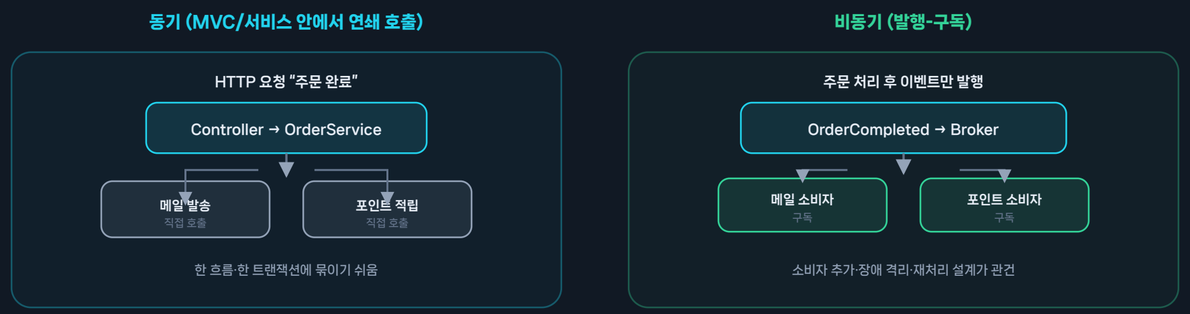
**생산자**는 이벤트 **스키마와 의미(계약)**를 책임집니다. **소비자**는 수신 **이후의 효과(Effect)**—다른 시스템 갱신, 알림, 보상 트랜잭션 등—에 책임을 집니다. **브로커 소유**는 **중앙 집중형**(거버넌스·표준)과 **분산형**(팀 자율)으로 갈리며, 운영 모델이 달라집니다.

01 · 05 연결

# MVC 흐름 vs 이벤트 기반 흐름

**MVC**는 전형적으로 **요청 → 컨트롤러 → 모델·뷰 → 응답**으로 이어지는 **동기식 요청-응답**입니다. 한 HTTP 요청 안에서 여러 부가 작업을 엮기 쉽습니다. **EDA**는 **발행-구독**으로 생산자가 사실(이벤트)만 브로커에 남기고, 소비자가 각자 비동기로 반응합니다. 앞선 EDA 슬라이드에서 본 **느슨한 결합·독립 배포**가 이 구조에서 나옵니다.

예: **주문 완료** 뒤 **메일 발송**, **포인트 적립** 같은 **부수 효과**(side effect)를 같은 서비스 메서드에 한꺼번에 호출하면 응답 시간·장애 전파 범위가 커집니다. 이벤트로 쪼개면 주문 처리는 빨리 끝내고, 알림·포인트는 별 소비자가 맡을 수 있습니다.



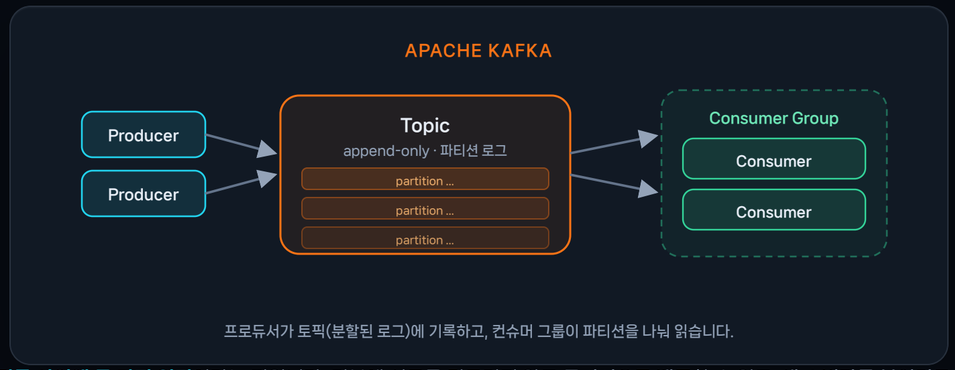

동기 연쇄는 단순해 보이지만 응답·실패가 한 줄에 묶입니다. 이벤트는 계약·멱등·순서를 따져야 하는 대신 확장과 분리에 유리합니다.

> 비동기로 쪼개면 요청 하나로 흐름이 끝나지 않아, **처리 상태·추적**이 어려워질 수 있습니다. 실무에서는 **분산 추적 도구(예: Zipkin, Jaeger)**와 **로그 통합·상관 ID**(trace id)로 "누가 어떤 이벤트를 어디까지 처리했는지"를 잇는 경우가 많습니다.

05 · KAFKA

# Kafka

**이벤트**는 **메시지의 한 형태**로 볼 수 있습니다. 전달에는 보통 메시징이 쓰이고, **Apache Kafka**는 그중에서도 이벤트 브로커로 널리 쓰이는 대표 사례입니다.




이벤트 기반 아키텍처의 이점 가운데 하나는 **생산자와 소비자가 직접 붙지 않고, 브로커를 사이에 둔 간접 연결**이라는 점입니다. 덕분에 서로를 건드리지 않고 **독립적으로 배포**할 수 있고, **새 소비자를 붙이기**도 수월합니다.

반면 이벤트가 항상 중간 브로커를 지나가므로, **처리 상태·흐름을 추적**하려면 **관측·추적 도구** 같은 별도 수단이 필요할 수 있습니다.

> **Kafka**는 로그형 토픽·파티션·컨슈머 그룹으로 **높은 처리량과 재처리**에 강하지만, 운영·스키마·정확히 한 번 처리 등은 설계와 운영 규율이 함께 가야 합니다.

05 · 브로커와 진실 공급원

# 이벤트 브로커와 단일 진실 공급원

이벤트 브로커를 **단일 진실 공급원(Single Source of Truth, SSOT)**처럼 확장해 쓰는 방식도 있습니다. 예를 들어 Kafka는 메시지를 곧바로 지우지 않고 **디스크에 유지**할 수 있어, 이 성질을 이용하면 생산자가 보낸 **이벤트를 오래 보관·재생**하는 데 유리합니다.

시스템의 모든 상태 변화가 **이벤트로만** 일어난다면, **이벤트 기록만으로도** 상태 변화를 따라가거나 재구성할 수 있습니다.

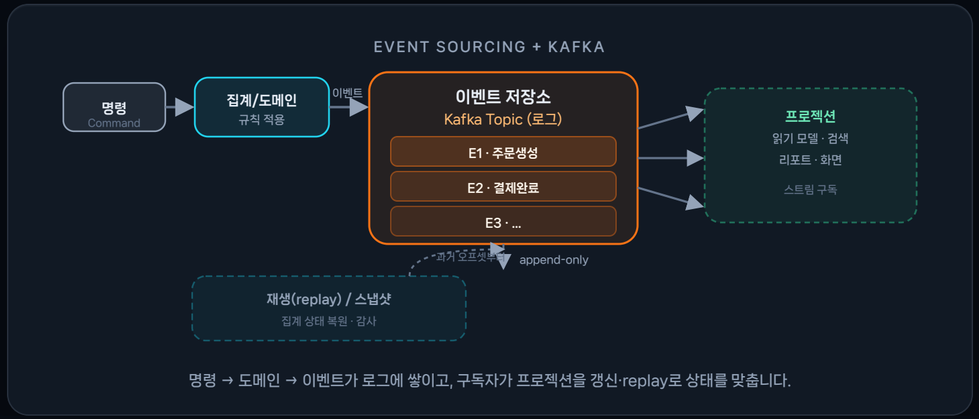
> SSOT·영구 로그로 쓸수록 **보관 기간·용량·개인정보·감사** 요건이 함께 따라옵니다. "남길 것"과 "지울 것"을 정책으로 정하는 것이 중요합니다.

05 · 이벤트 소싱과 저장소

# 이벤트 소싱에서 Kafka를 이벤트 저장소로

**이벤트 소싱**은 행 하나만 덮어쓰는 대신 **도메인 이벤트를 시간 순으로** append하고, 스트림을 replay해 과거 시점 상태까지 재구성합니다.

**Kafka 토픽**은 파티션·오프셋으로 **순서가 잡힌 로그**라 후보로 자주 쓰이지만, DB 이벤트 스토어와는 트랜잭션·중복·스키마 요구가 달라 **멱등 소비·보존**(compaction)을 함께 설계합니다. 같은 스트림으로 **프로젝션**을 돌리면 CQRS와도 맞물립니다.



05 · 06 연결

# 동기 API vs 이벤트 메시징

Kafka·브로커로 이어지는 **비동기 전달**과, REST·RPC처럼 **바로 응답을 받는 동기 호출**은 트레이드오프가 다릅니다. 동기는 호출자가 즉시 성공·실패를 알 수 있어 **강한 일관성에 가깝게** 설계하기 쉽고, 이벤트는 전파 지연이 허용될 때 **결과적 일관성(eventual consistency)**으로 맞추는 경우가 많습니다.

### 동기 API

- 호출 체인이 명확하고 디버깅이 직관적
- 상대가 느리거나 죽으면 내 응답도 막힘
- 2단계 커밋 등으로 분산 일관성을 비싸게 살 수 있음

### 이벤트 메시징

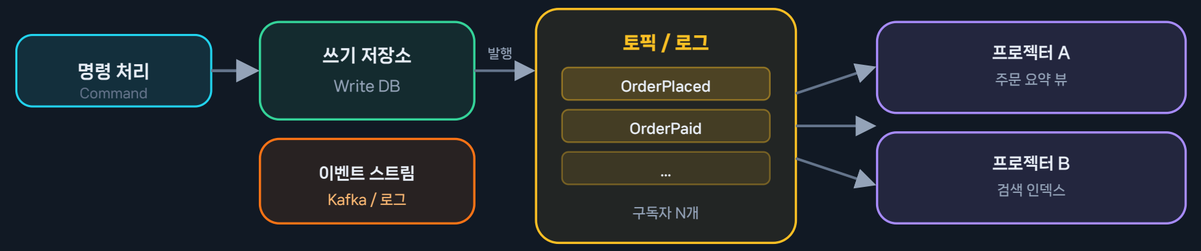
- 생산자·소비자를 시간·버전으로 분리
- 중복·순서·지연을 전제로 멱등·보상 설계
- SSOT 로그·재처리와 궁합이 좋음

**CQRS**에서 쓰기 DB와 읽기 DB(또는 검색 인덱스)를 나누면, 읽기 쪽을 맞추기 위해 **이벤트를 구독해 동기화**하는 일이 생깁니다. 앞서 표로 정리한 단점인 **"데이터 동기화를 위한 메시징 등 추가 수단"**이 바로 이 축입니다. 동기 API만으로 읽기 모델을 갱신할 수도 있지만, 트래픽·결합 조건에 따라 이벤트가 더 낫습니다.

05 · 06 연결

# 프로젝션: 이벤트 → 조회 전용 모델

명령 쪽에서 상태를 바꾸면 **도메인 이벤트**가 흐르고, **조회(Query) 모델**은 그 스트림을 **소비(consume)**하면서 화면·검색에 맞는 형태로 **재구성(project)**합니다. 같은 이벤트를 여러 프로젝터가 각기 다른 읽기 저장소에 반영할 수 있어, CQRS의 "읽기 최적화"와 맞닿습니다.




이벤트 순서·중복·스키마 진화를 프로젝터가 흡수해야 읽기 모델이 안정적입니다.

조회 전용 저장소는 API로 쓰기 DB를 두드리는 방식과 병행할 수도 있고, 이벤트만으로 따라가게 만들 수도 있습니다. 운영·일관성 요구에 맞게 고릅니다.

**CQRS·이벤트 소싱·프로젝션**은 모든 시스템에 필수는 아닙니다. **단순한 모델**에 억지로 얹으면 **공수·운영 부담만 커질 수 있고**, **복잡해질수록 분리 이득이 큰지**를 따져 도입 여부를 판단하는 것이 중요합니다.

06

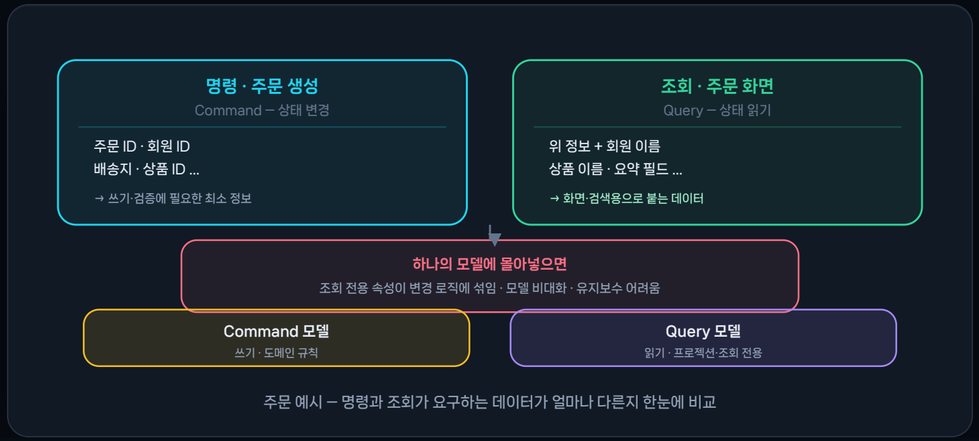
# CQRS

명령과 조회의 책임 분리

06 · 정의 · 범위 · 결합

# CQRS

명령은 상태를 바꾸고 조회는 읽습니다. 단순할 땐 같은 모델로도 되지만, 복잡해지면 둘이 쓰는 데이터가 달라집니다. 아래 주문 예처럼 생성에는 최소 식별자만 있으면 되고, 화면 조회에는 이름·상품명처럼 붙는 필드가 더 필요합니다. 그런데도 하나의 모델에 몰아넣으면 조회용 속성이 변경 코드에 섞여 비대해집니다. 그때 CQRS(Command Query Responsibility Segregation)로 쓰기 모델과 읽기 모델을 나눕니다.




주문 예시 — 명령과 조회가 요구하는 데이터가 얼마나 다른지 한눈에 비교

# CQRS의 이점과 단점

| 구분 | 정리 |
| --- | --- |
| 이점 | **명령·조회 모델 분리**로 기능에 맞게 모델을 구현할 수 있습니다. **단일 모델**이면 조회 때문에 명령이 흔들리거나 그 반대가 될 수 있는데, 분리하면 **상호 영향**을 줄입니다. |
| | **조회 모델**이 따로 있으면 **읽기 성능**을 올리기 쉽습니다. **캐시**를 두거나 **조회 전용** DB를 확장하는 식이 대표적입니다. |
| 단점 | 기능마다 모델을 만들어야 해 **코드량**이 늘고, **추가 코드 대비 이득**을 따져야 합니다. **단순 모델**에 CQRS를 얹으면 공수만 커질 수 있고, **복잡한 모델**일수록 이득이 클 가능성이 높습니다. |
| | **구현 기술**이 늘 수 있습니다. 특히 조회 모델을 **별도 읽기** DB에 두면 **데이터 동기화**를 위해 **메시징** 등 추가 수단이 필요해질 수 있습니다. |

총정리

# 발표 내용 되짚기

### 01 · MVC
Model · View · Controller로 역할을 나누고, 요청이 컨트롤러를 거쳐 모델·뷰로 흐르게 합니다.

### 02 · 계층형
**표현 → 응용 → 도메인 → 인프라**로 나누고, 도메인에 비즈니스 규칙을 모읍니다. 계층 간 의존 방향을 지킵니다.

### 03 · DDD
**엔티티·밸류·애그리거트·리포지토리·도메인 이벤트**로 모델을 다듬고, Bounded Context로 말이 통하는 경계를 잡습니다.

### 04 · MSA
도메인·배포 단위로 서비스를 쪼개 **독립 배포·자체 상태**를 갖추되, 분산 비용·운영 부담은 함께 따릅니다.

### 05 · EDA · Kafka
**생산자 · 브로커 · 소비자**로 이벤트를 주고받아 결합을 느슨하게 합니다. Kafka·SSOT·이벤트 소싱은 요구에 맞게 조합합니다.

### 06 · CQRS
**명령 모델**과 **조회 모델**을 나눠 상호 영향과 모델 비대를 줄이고, 조회 전용 확장(캐시·읽기 DB)을 열어 둡니다. 트레이드오프를 봅니다.

마무리

# 감사합니다

질문 환영합니다.